SYSTEM ARCHITECTURE NOTE

TASK TITLE: Dhruv Patel, Multi-Sensor Data Pipeline + System Architecture (Robotics Systems – Test 1) / March-2026

1.PURPOSE

This document defines the system architecture for a Multi-Sensor data pipeline. Three sensors — an IMU (acceleration + orientation), Depth/Distance sensor (range-based), and Contact sensors (foot-level binary) are integrated into a single failure-aware, time-synchronized system. The pipeline is built as a standalone prototype but structured to plug directly into a real robotic platform in the future.

2. SENSOR OVERVIEW

SENSOR	DATA RATE	DATA OUTPUT	FAILURE CONDITIONS
IMU (Inertial Measurement Unit)	10 Hz (High Frequency)	Acceleration[x,y,z] m/s^2 Orientation [roll, pitch, yaw](rad)	Gyroscope drift over time, Gaussian noise on all axes, random spike values, packet loss
Distance/Depth Sensor (Range based)	2 Hz (Medium Frequency)	Single float — range in metres Valid range: 0.05 m to 4.0 m	Out-of-range readings (99.9m),reflective errors, delayed packets
Contact Sensors (Foot-level binary)	Event-based (on change only)	Boolean:True = Dry / False=Leak+Timestamp	Stuck state, vibration jitter, missed events at high speeds

3. SENSOR DETAIL

- **IMU — the most critical sensor** The most critical sensor. It runs at 10 Hz, making it the timing reference for the entire pipeline; all other sensors sync to it.
- **Distance Sensor — terrain awareness.** It fires at 2 Hz, which is 5x slower than the IMU. For every depth reading, the IMU produces 5 readings.

- **Leak Sensors:** Internal state tracking. Unlike the others, these do not produce continuous data. They only send a signal if a leak is detected or cleared. The pipeline stores the last known state and updates it only when a new event arrives.

---

4. The 4-Step Data Flow

1. **Ingestion:** Processes raw CSV files with unsynchronized timestamps from the IMU and Depth sensors.
2. **Synchronization:** Performs a **Timestamp Handshake**. For every fast IMU reading, it finds the "Closest Neighbor" in the slow Depth data to create a unified packet.
3. **Validation:** Applies a **Threshold Rule** (Is Depth > 50m?). Spikes are tagged as CRITICAL, and normal data as HEALTHY.
4. **Visualization:** Generates a **Scatter Plot (S vs T)** where Blue dots show valid data and Red dots highlight failures.

---

5. Failure scenarios handled

- **Sensor Spike (99.9m):** Caused by electrical noise; handled by a **Threshold Filter** ($>50m$) to prevent "Ghost Dives".
- **Temporal Drift:** Caused by different sampling rates; handled by **Nearest-Neighbor Matching**.
- **Data Loss:** Caused by out-of-phase hardware; handled by **Status Tagging** (CRITICAL) to trigger fail-safe modes.

---

6. Assumptions (The "Given" Conditions)

- **Linear Time:** We assume the system clock is synchronized and timestamps always move forward.
- **Sampling Consistency:** We assume the IMU always stays at 10Hz and Depth at 2Hz without "jitter" (varying speeds).
- **Static Threshold:** We assume 50m is a hard physical limit for the AUV's current mission environment.
- **File Availability:** We assume the CSV files are not corrupted and are accessible by the Python script.

---

7. Limitations (The "Future" Fixes)

- **Offline Processing:** The current architecture is "Batch" (reads files) rather than "Real-Time" (streaming live data).
- **Single-Point Failure:** If the Depth sensor stops sending data entirely (0 Hz), the sync logic will keep pairing the *last* known depth to new IMU data indefinitely.
- **Hard-Coded Logic:** The 50m anomaly threshold is "hard-coded." In a real mission, this should be "dynamic" based on the seafloor map.
- **No Smoothing:** The detector flags spikes but doesn't "filter" them out (like a Kalman Filter or Moving Average would) to keep the data stream smooth.

"Clarity of architecture matters more than volume of code."